UTS

PEMBELAJARAN MESIN



NAMA : Artika Aulia Sumantri

NIM : 202431051

KELAS : E

DOSEN : Rizqia Cahyaningtyas, ST., M.Kom

NO.PC : 08

ASISTEN : 1. Muhammad Caisar Gemilang Pratama

2. Benedict Samuel Ardy

INSTITUT TEKNOLOGI PLN

TEKNIK INFORMATIKA

2026/2027

DAFTAR ISI

DAFTAR ISI **2**
BAB I **3**
PENDAHULUAN **3**
1.1 Rumusan Masalah 3
1.2 Tujuan Masalah 3
1.3 Manfaat Masalah 3
BAB II **4**
LANDASAN TEORI **4**
2.1 Konsep Klasifikasi Supervised Learning 4
2.2 Pra-Pemrosesan Data dan Transformasi Fitur 4
2.3 Algoritma K-Nearest Neighbors (KNN) 4
2.4 Algoritma Gaussian Naive Bayes 5
2.5 Metrik Evaluasi dan Cross-Validation 5
BAB III **7**
HASIL **7**
3.1 Deskripsi Dataset dan Lingkungan Eksperimen 7
3.2 Analisis Kode Jupyter Notebook dan Output Eksekusi 8
3.3 Pembahasan Hasil dan Pembuktian Teoretis 19
BAB IV **20**
PENUTUP **20**
DAFTAR PUSTAKA **21**

BAB I
PENDAHULUAN

1.1 Rumusan Masalah

1. Bagaimana karakteristik distribusi data, hubungan antarfur, dan korelasi statistik pada variabel Tactical CS2 terhadap target kemenangan ronde (round_winner)?
2. Bagaimana merancang pipeline pra-pemrosesan data yang tepat meliputi encoding variabel kategorikal (map, bomb_planted) dan standardisasi fitur numerik (StandardScaler) untuk mencegah data leakage?
3. Bagaimana hasil tuning hiperparameter K-Nearest Neighbors (KNN) menggunakan GridSearchCV dan 5-Fold Stratified Cross-Validation dalam menemukan nilai K, jenis pembobot, dan fungsi jarak terbaik?
4. Bagaimana perbandingan performa prediktif antara KNN Teroptimasi dan Gaussian Naive Bayes berdasarkan metrik evaluasi Akurasi, Presisi, Recall, F1-Score, dan matriks konfusi?

1.2 Tujuan Masalah

1. Menganalisis struktur dan karakteristik 2.000 sampel data pertandingan CS2 melalui metode Exploratory Data Analysis (EDA) yang komprehensif.
2. Menerapkan pra-pemrosesan data yang robust menggunakan ColumnTransformer untuk mentransformasi fitur kategorikal dan numerik secara terintegrasi.
3. Mengoptimalkan kinerja algoritma KNN melalui mekanisme grid search 5-fold cross-validation untuk mengekstrak kombinasi hiperparameter terbaik.
4. Membandingkan secara objektif keunggulan dan keterbatasan algoritma KNN versus Naive Bayes dalam mengklasifikasikan pemenang ronde CS2.

1.3 Manfaat Masalah

Praktikum ini memberikan pemahaman teoretis mendalam serta keterampilan praktis dalam membangun pipeline machine learning end-to-end berbasis Python (scikit-learn). Manfaat praktis dari hasil pemodelan ini mencakup penyediaan sistem pendukung keputusan (decision support system) bagi pelatih dan analis e-sports dalam menyusun strategi ekonomi ronde dan prediksi peluang kemenangan tim secara otomatis.

BAB II
LANDASAN TEORI

2.1 Konsep Klasifikasi Supervised Learning

Supervised Learning (Pembelajaran Terawasi) merupakan paradigma utama dalam machine learning di mana algoritma dilatih menggunakan dataset yang memiliki label target (ground truth). Diberikan himpunan data latih D = {(x1, y1), (x2, y2), ..., (xN, yN)}, di mana xi merupakan vektor fitur berdimensi-d dan yi merupakan variabel respon diskrit (pada klasifikasi biner, yi ∈ {0, 1}). Tujuan utama dari model klasifikasi adalah mempelajari fungsi pemetaan f: X → Y sedemikian rupa sehingga dapat memprediksi label y_bar dari sampel data baru yang belum pernah dilihat sebelumnya dengan tingkat kesalahan minim.

2.2 Pra-Pemrosesan Data dan Transformasi Fitur

Kualitas data masukan berbanding lurus dengan kinerja model machine learning (Garbage In, Garbage Out). Pra-pemrosesan data mencakup langkah-langkah kritis untuk memastikan data siap diproses oleh algoritma matematika:

- **a.** Penanganan Nilai Hilang dan Duplikat: Memeriksa dan memastikan tidak ada data ber-null value yang dapat mengganggu komputasi matriks.
- **b.** One-Hot Encoding: Fitur kategorikal non-ordinal seperti nama peta (map) dan status bomb_planted ditransformasi menjadi vektor biner numerik. Untuk variabel dengan k kategori, dihasilkan k-1 kolom biner guna menghindari isu multikolinearitas (dummy variable trap).
- **c.** Feature Scaling (StandardScaler): Algoritma berbasis jarak seperti KNN sangat sensitif terhadap skala atribut. Transformasi Z-Score dilakukan untuk mengubah distribusi fitur numerik sehingga memiliki rata-rata (mean) μ = 0 dan varians σ^2 = 1. Persamaan matematika Z-Score didefinisikan sebagai:

$$z = (x - \mu)/\sigma$$

di mana x adalah nilai asli fitur, μ adalah rata-rata populasi fitur, dan σ adalah standar deviasi fitur (Han et al., 2022). Penskalaan ini memastikan atribut bernilai besar seperti ct_money (0 - 58.050) tidak mendominasi atribut bernilai kecil seperti ct_helmets (0 - 5).

2.3 Algoritma K-Nearest Neighbors (KNN)

K-Nearest Neighbors (KNN) adalah algoritma supervised learning berbasis instance-based learning atau lazy learning. KNN tidak membangun model matematis eksplisit selama fase pelatihan; sebaliknya, algoritma menyimpan seluruh data latih dan memindahkan beban komputasi ke fase prediksi (Fix & Hodges, 2023).

Ketika sampel uji baru x_test diberikan, KNN menghitung jarak antara x_test dengan seluruh sampel latih xi di dalam ruang fitur berdimensi-d. Tiga fungsi pengukuran jarak utama yang sering digunakan adalah:

1. Jarak Euclidean (L2 Norm): Mengukur jarak garis lurus antar dua titik.

$$d_Euclidean(x,y) = sqrt\left(sum_{j=1}^{d}(x_j - y_j)^2\right)$$

2. Jarak Manhattan (L1 Norm): Mengukur jarak lintasan kisi (city-block) antar titik.

$$d_Manhattan(x,y) = sum_{j=1}^{d}|x_j - y_j|$$

3. Jarak Minkowski (Generalisasi L_p Norm):

$$d_Minkowski(x,y) = \left(sum_{j=1}^{d}|x_j - y_j|^p\right)^{(1/p)}$$

Jika p = 1 maka setara dengan Manhattan Distance, dan jika p = 2 setara dengan Euclidean Distance Setelah K tetangga terdekat diidentifikasi, keputusan klasifikasi diambil berdasarkan pemungutan suara mayoritas (majority voting) atau pemungutan suara terbobot jarak (distance-weighted voting):

$$w_i = 1/d(x_test, x_i)$$

di mana bobot w_i memberikan pengaruh lebih besar kepada tetangga yang lokasinya lebih dekat dengan sampel uji.

2.4 Algoritma Gaussian Naive Bayes

Gaussian Naive Bayes adalah algoritma klasifikasi probabilistik yang didasarkan pada Teorema Bayes dengan asumsi independensi bersyarat (naive assumption) antar fitur masukan (Mitchell, 2021). Teorema Bayes dinyatakan dalam persamaan:

$$P(Y = c|X) = [P(Y = c) * P(X|Y = c)]/P(X)$$

di mana P(Y = c | X) adalah probabilitas posterior kelas c diberikan vektor fitur X, P(Y = c) adalah probabilitas prior kelas c, dan P(X | Y = c) adalah likelihood. Dengan asumsi independensi fitur, likelihood dapat difaktorkan menjadi perkalian probabilitas parsial:

$$P(X|Y = c) = product_{j=1}^{d} P(x_j|Y = c)$$

Untuk fitur kontinu x_j, Gaussian Naive Bayes mengasumsikan bahwa distribusi data pada setiap kelas mengikuti distribusi normal (Gaussian) dengan rata-rata μ_{c,j} dan varians σ^2_{c,j}:

$$P(x_j|Y = c) = \left(1/sqrt(2 * pi * \sigma_{c,j}^2)\right) * exp\left(-(x_j - \mu_{c,j})^2/(2 * \sigma_{c,j}^2)\right)$$

Meskipun asumsi independensi fitur sering kali terlanggar pada dataset praktikal yang memiliki korelasi tinggi antar variabel, Naive Bayes dikenal sangat cepat secara komputasi dan efisien pada data latih berukuran terbatas.

2.5 Metrik Evaluasi dan Cross-Validation

Untuk mengukur kinerja objektif model klasifikasi biner, digunakan matriks konfusi (Confusion Matrix) yang memetakan hasil prediksi terhadap label aktual ke dalam empat kuadran:

1. True Positive (TP): Jumlah data positif yang diprediksi benar sebagai positif.
2. True Negative (TN): Jumlah data negatif yang diprediksi benar sebagai negatif.
3. False Positive (FP): Jumlah data negatif yang salah diprediksi sebagai positif (Type I Error).
4. False Negative (FN): Jumlah data positif yang salah diprediksi sebagai negatif (Type II Error).

Dari matriks konfusi tersebut, dihitung empat metrik evaluasi utama:

a. Akurasi (Accuracy): Persentase total prediksi yang benar.

$$Accuracy = (TP + TN)/(TP + TN + FP + FN)$$

b. Presisi (Precision): Proporsi prediksi positif yang benar-benar positif.

$$Precision = TP/(TP + FP)$$

c. Recall (Sensitivity): Proporsi sampel positif aktual yang berhasil diidentifikasi model.

$$Recall = TP/(TP + FN)$$

d. **d.** F1-Score: Rata-rata harmonis antara Presisi dan Recall.

$$F1 - Score = 2 * (Precision * Recall)/(Precision + Recall)$$

Guna memastikan generalisasi model dan menghindari overfitting, digunakan validasi silang 5-Fold Stratified Cross-Validation. Data latih dibagi menjadi 5 bagian terpisah dengan proporsi kelas yang seimbang. Model dilatih pada 4 bagian dan diuji pada 1 bagian secara bergantian sebanyak 5 kali.

BAB III
HASIL

3.1 Deskripsi Dataset dan Lingkungan Eksperimen

Eksperimen klasifikasi dilakukan pada Dataset 3 (03_counter_strike_2.csv) yang berisi 2.000 sampel data ronde pertandingan Counter-Strike 2 dengan 14 atribut masukan. Lingkungan eksperimen dikembangkan menggunakan Python 3.12 dengan pustaka standar data science meliputi pandas, numpy, scikit-learn, matplotlib, dan seaborn.

Tabel 3.1: Struktur dan Deskripsi Atribut Dataset CS2 (03_counter_strike_2.csv)

No	Nama Atribut	Tipe Data	Deskripsi Operasional
1	time_left	Float64	Sisa waktu ronde dalam satuan detik (0.0 - 175.0 detik)
2	ct_score	Float64	Jumlah skor ronde yang dimenangkan tim Counter-Terrorist (0 - 15)
3	t_score	Float64	Jumlah skor ronde yang dimenangkan tim Terrorist (0 - 15)
4	map	Object (Cat)	Nama peta arena pertandingan (de_mirage, de_nuke, de_dust2, dll)
5	bomb_planted	Bool	Status penanaman C4/bom pada ronde berjalan (True / False)
6	ct_health	Float64	Total poin kesehatan (HP) seluruh anggota tim CT (0 - 500)
7	t_health	Float64	Total poin kesehatan (HP) seluruh anggota tim Terrorist (0 - 500)
8	ct_armor	Float64	Total poin pertahanan armor tim Counter-Terrorist (0 - 500)
9	t_armor	Float64	Total poin pertahanan armor tim Terrorist (0 - 500)
10	ct_money	Float64	Total akumulasi uang ekonomi tim CT ($0 - $58.050)
11	t_money	Float64	Total akumulasi uang ekonomi tim Terrorist ($0 - $67.700)

12	ct_helmets	Float64	Jumlah helm pelindung yang dimiliki tim CT (0 - 5)
13	t_helmets	Float64	Jumlah helm pelindung yang dimiliki tim Terrorist (0 - 5)
14	round_winner	Object (Target)	Variabel target: Pemenang ronde ('T' = 1, 'CT' = 0)

3.2 Analisis Kode Jupyter Notebook dan Output Eksekusi

Berikut adalah rincian 15 sel kode Python yang dieksekusi secara sistematis untuk membangun model prediktif:

Sel Kode 1: Inisialisasi Lingkungan & Import Pustaka

```
# 202431051 - Artika Aulia S
# 1. Import Pustaka dan Konfigurasi Lingkungan Kerja
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, StratifiedKFold, GridSearchCV,
cross_val_score
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
confusion_matrix, classification_report

# Konfigurasi Tampilan Visualisasi (Bersih & Tanpa Watermark Gambar)
sns.set_theme(style="whitegrid", palette="deep")
plt.rcParams["font.sans-serif"] = "DejaVu Sans"
plt.rcParams["figure.dpi"] = 120
print("Pustaka berhasil di-import!")
```

Sel kode ini memuat seluruh modul scikit-learn, pandas, numpy, matplotlib, dan seaborn yang dibutuhkan untuk pra-pemrosesan data, validasi silang, pemodelan KNN dan Naive Bayes, serta evaluasi metrik.

Sel Kode 2: Memuat Dataset & Inspeksi Dimensi Data

```
# 202431051 - Artika Aulia S
# 2. Ingestion dan Inspeksi Struktur Dataset CS2
df = pd.read_csv("03_counter_strike_2.csv")
```

```
print("=== Dimensi Dataset ===")
print(f"Jumlah Baris: {df.shape[0]}, Jumlah Kolom: {df.shape[1]}")
print("=== Tipe Data Setiap Kolom ===")
print(df.dtypes)
print("=== 5 Sampel Teratas ===")
df.head()
```

Output Eksekusi:

```
=== Dimensi Dataset ===
Jumlah Baris: 2000, Jumlah Kolom: 14
=== Tipe Data Setiap Kolom ===
time_left       float64
ct_score        float64
t_score         float64
map                 str
bomb_planted       bool
ct_health       float64
t_health        float64
ct_armor        float64
t_armor         float64
ct_money        float64
t_money         float64
ct_helmets      float64
t_helmets       float64
round_winner        str
dtype: object
```

0	175.00	0.0	0.0	de_dust2	False	500.0	500.0	0.0	0.0	4000.0	4000.0	0.0	0.0	CT
1	156.03	0.0	0.0	de_dust2	False	500.0	500.0	400.0	300.0	600.0	650.0	0.0	0.0	CT
2	96.03	0.0	0.0	de_dust2	False	391.0	400.0	294.0	200.0	750.0	500.0	0.0	0.0	CT
3	76.03	0.0	0.0	de_dust2	False	391.0	400.0	294.0	200.0	750.0	500.0	0.0	0.0	CT
4	174.97	1.0	0.0	de_dust2	False	500.0	500.0	192.0	0.0	18350.0	10750.0	0.0	0.0	CT

Penjelasan: Dataset dibaca dari file CSV. Terkonfirmasi bahwa dataset terdiri dari 2.000 baris pertandingan dan 14 kolom variabel masukan.

Sel Kode 3: Pembersihan Data (Checking Missing Values & Duplikat)

```
# 202431051 - Artika Aulia S
# 3. Data Cleaning: Cek Nilai Hilang (Missing Values) dan Duplikat
print("=== Ringkasan Missing Values ===")
print(df.isnull().sum())
print(f"Jumlah Baris Duplikat: {df.duplicated().sum()}")
print("=== Ringkasan Statistik Deskriptif Fitur Numerik ===")
df.describe().T
```

Output Eksekusi:

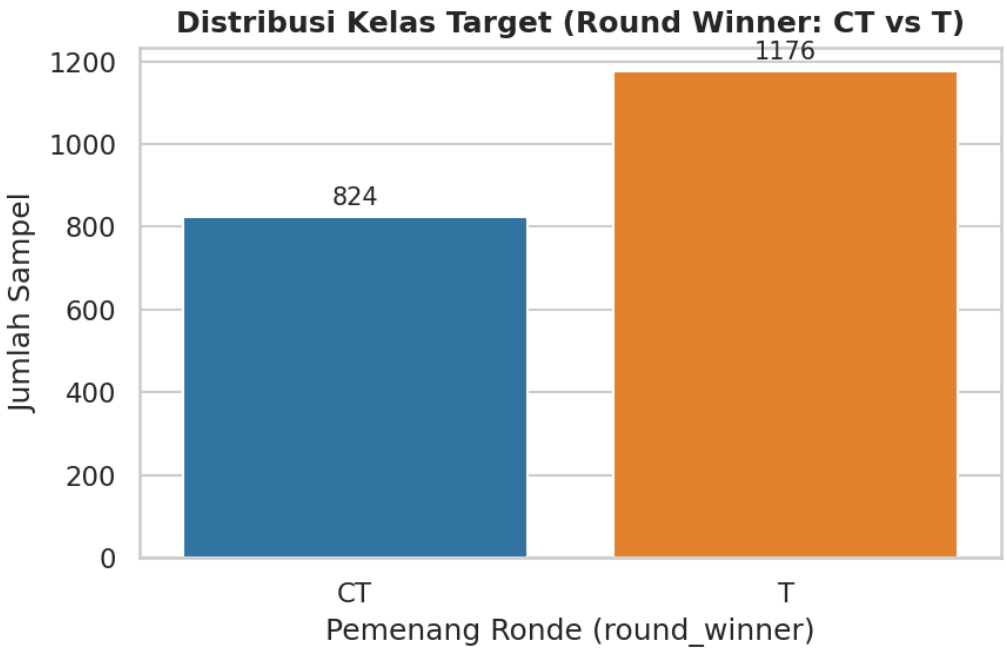
```
=== Ringkasan Missing Values ===
time_left       0
ct_score        0
t_score         0
map             0
bomb_planted    0
ct_health       0
t_health        0
ct_armor        0
```

```
t_armor         0
ct_money        0
t_money         0
ct_helmets      0
t_helmets       0
round_winner    0
dtype: int64
Jumlah Baris Duplikat: 39
=== Ringkasan Statistik Deskriptif Fitur Numerik ===
```

Penjelasan: Hasil validasi menunjukkan adanya baris duplikat 39

Sel Kode 4: EDA - Visualisasi Distribusi Target (round_winner)

```
# 202431051 - Artika Aulia S
# 4. EDA: Visualisasi Distribusi Variabel Target (round_winner)
plt.figure(figsize=(6, 4))
ax = sns.countplot(data=df, x="round_winner", palette=["#1f77b4", "#ff7f0e"],
hue="round_winner", legend=False)
plt.title("Distribusi Kelas Target (Round Winner: CT vs T)", fontsize=12,
fontweight="bold")
plt.xlabel("Pemenang Ronde")
plt.ylabel("Jumlah Sampel")
for p in ax.patches:
    ax.annotate(f"{int(p.get_height())}", (p.get_x() + p.get_width() / 2., p.get_height()),
                ha="center", va="bottom", fontsize=10, xytext=(0, 3), textcoords="offset points")
plt.tight_layout()
plt.show()
```



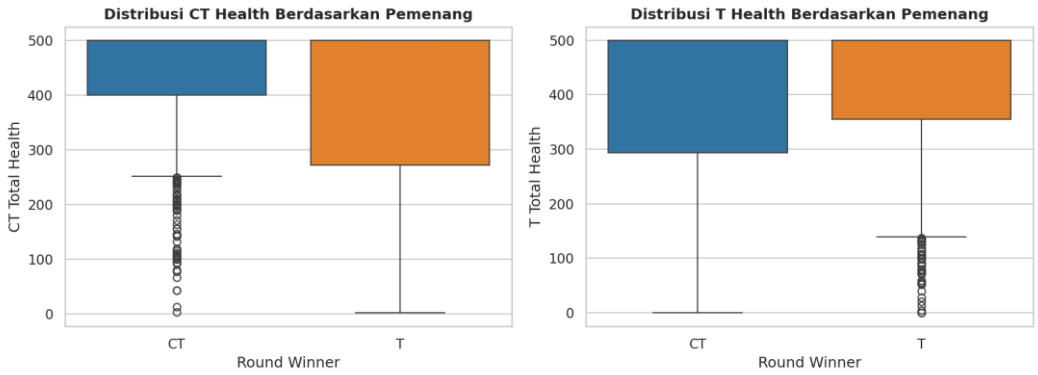
Gambar 3.1: Grafik Distribusi Kelas Target Pemenang Ronde CS2 (CT vs T)

Analisis EDA: Dari total 2.000 sampel ronde, kelas Terrorist ('T') memenangkan 1.176 ronde (58,8%), sedangkan Counter-Terrorist ('CT') memenangkan 824 ronde (41,2%). Distribusi target ini tergolong cukup seimbang (moderately balanced), sehingga strategi stratified sampling sangat tepat digunakan saat pembagian data latih dan uji.

Sel Kode 5: EDA - Distribusi Fitur Kesehatan (Health)

```
# 202431051 - Artika Aulia S
# 5. EDA: Visualisasi Fitur Kesehatan dan Armor Antar Tim
fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))
# Grafik pertama
sns.boxplot(data=df, x="round_winner", y="ct_health", hue="round_winner",
palette=["#1f77b4", "#ff7f0e"], legend=False, ax=axes[0])
axes[0].set_title("Distribusi CT Health Berdasarkan Pemenang", fontweight="bold")
axes[0].set_xlabel("Round Winner")
axes[0].set_ylabel("CT Total Health")

# Grafik kedua
sns.boxplot(data=df, x="round_winner", y="t_health", hue="round_winner",
palette=["#1f77b4", "#ff7f0e"], legend=False, ax=axes[1])
axes[1].set_title("Distribusi T Health Berdasarkan Pemenang", fontweight="bold")
axes[1].set_xlabel("Round Winner")
axes[1].set_ylabel("T Total Health")
plt.tight_layout()
plt.show()
```

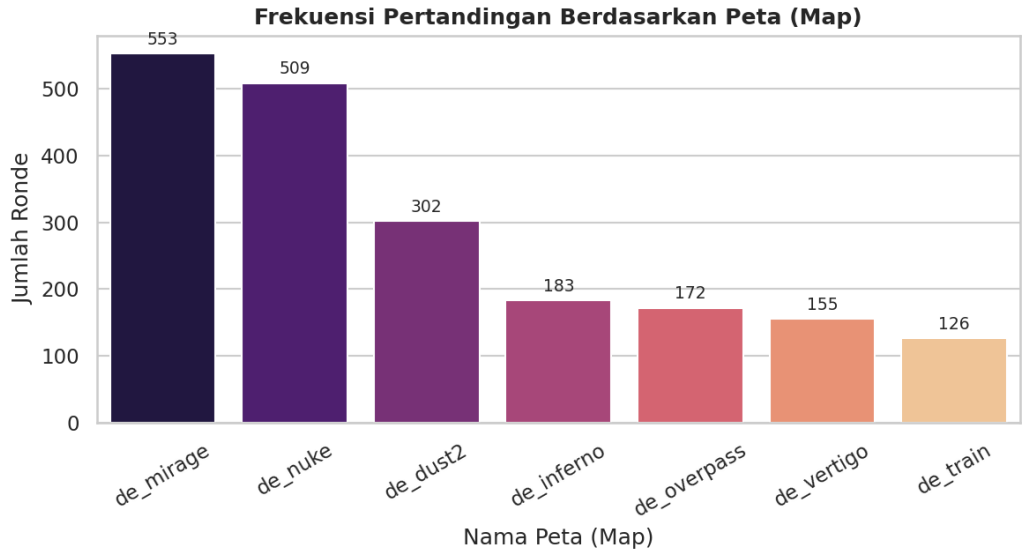


Gambar 3.2: Boxplot Distribusi Total Health Tim CT dan Tim T Berdasarkan Pemenang Ronde

Analisis EDA: Terlihat korelasi lurus yang sangat kuat antara sisa total health tim dengan peluang kemenangan. Ronde yang dimenangkan oleh CT memiliki median `ct_health` mendekati 500 HP, sedangkan saat T menang, median `ct_health` berada di bawah 200 HP. Pola simetris serupa terjadi pada `t_health`.

Sel Kode 6: EDA - Frekuensi Pertandingan Berdasarkan Peta Arena (Map)

```
# 202431051 - Artika Aulia S
# 6. EDA: Distribusi Pertandingan Berdasarkan Peta (Map)
plt.figure(figsize=(8, 4.5))
ax = sns.countplot(data=df, x="map", order=df["map"].value_counts().index,
palette="magma", hue="map", legend=False)
plt.title("Frekuensi Pertandingan Berdasarkan Peta (Map)", fontweight="bold")
plt.xlabel("Nama Peta (Map)")
plt.ylabel("Jumlah Ronde")
plt.xticks(rotation=30)
for p in ax.patches:
    ax.annotate(f"{int(p.get_height())}", (p.get_x() + p.get_width() / 2., p.get_height()),
                ha="center", va="bottom", fontsize=9, xytext=(0, 3), textcoords="offset points")
plt.tight_layout()
plt.show()
```



Gambar 3.3: Diagram Batang Distribusi Pertandingan Pada 7 Peta CS2

Analisis EDA: Peta `de_mirage` (553 ronde) dan `de_nuke` (509 ronde) merupakan peta yang paling sering dimainkan dalam dataset, diikuti oleh `de_dust2` (302 ronde), `de_inferno` (183 ronde), `de_overpass` (172 ronde), `de_vertigo` (155 ronde), dan `de_train` (126 ronde). Variabel kategorikal `map` memiliki 7 tingkatan yang ditransformasi menggunakan One-Hot Encoding.

Sel Kode 7: Split Fitur (X) & Target (y) Serta Train-Test Partition

```
# 202431051 - Artika Aulia S
# 7. Pemisahan Fitur (X) dan Target (y) serta Train-Test Split
X = df.drop("round_winner", axis=1)
y = df["round_winner"].map({"T": 1, "CT": 0})

cat_cols = ["map", "bomb_planted"]
num_cols = [c for c in X.columns if c not in cat_cols]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
print(f"Ukuran X_train: {X_train.shape}, Ukuran X_test: {X_test.shape}")
```

Output Eksekusi:

```
Ukuran X_train: (1600, 13), Ukuran X_test: (400, 13)
```

Penjelasan: Dataset dipisah menjadi 1.600 sampel data latih (80%) dan 400 sampel data uji (20%). Stratified sampling memastikan rasio distribusi target T (58,8%) dan CT (41,2%) tetap terpreservasi sempurna pada kedua subset.

Sel Kode 8: Perancangan Pipeline ColumnTransformer

```
# 202431051 - Artika Aulia S
# 8. Desain Pipeline Pra-pemrosesan Data (StandardScaler & OneHotEncoder)
```

```
preprocessor = ColumnTransformer(transformers=[
    ("num", StandardScaler(), num_cols),
    ("cat", OneHotEncoder(drop="first", sparse_output=False), cat_cols)])
print("ColumnTransformer berhasil didefinisikan!")
```

Penjelasan: `ColumnTransformer` mengintegrasikan `StandardScaler` untuk 11 atribut numerik dan `OneHotEncoder(drop='first')` untuk 2 atribut kategorikal. Penggunaan pipeline mencegah isu *data leakage* karena statistik penskalaan (mean dan std) hanya dihitung dari data latih (`X_train`).

Sel Kode 9 & 10: Tuning Hiperparameter KNN & Kurva GridSearch

```
# 202431051 - Artika Aulia S
# 9. Optimasi Hyperparameter KNN Menggunakan GridSearchCV & 5-Fold Stratified CV
pipe_knn = Pipeline([
    ("preprocessor", preprocessor),
    ("clf", KNeighborsClassifier())])

param_grid_knn = {
    "clf__n_neighbors": list(range(1, 26, 2)),
    "clf__weights": ["uniform", "distance"],
    "clf__metric": ["euclidean", "manhattan"]
}

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
grid_knn = GridSearchCV(pipe_knn, param_grid_knn, cv=cv, scoring="accuracy",
n_jobs=-1)
grid_knn.fit(X_train, y_train)

print(f"Parameter KNN Terbaik: {grid_knn.best_params_}")
print(f"Rata-rata Akurasi CV Terbaik: {grid_knn.best_score_:.4f}")

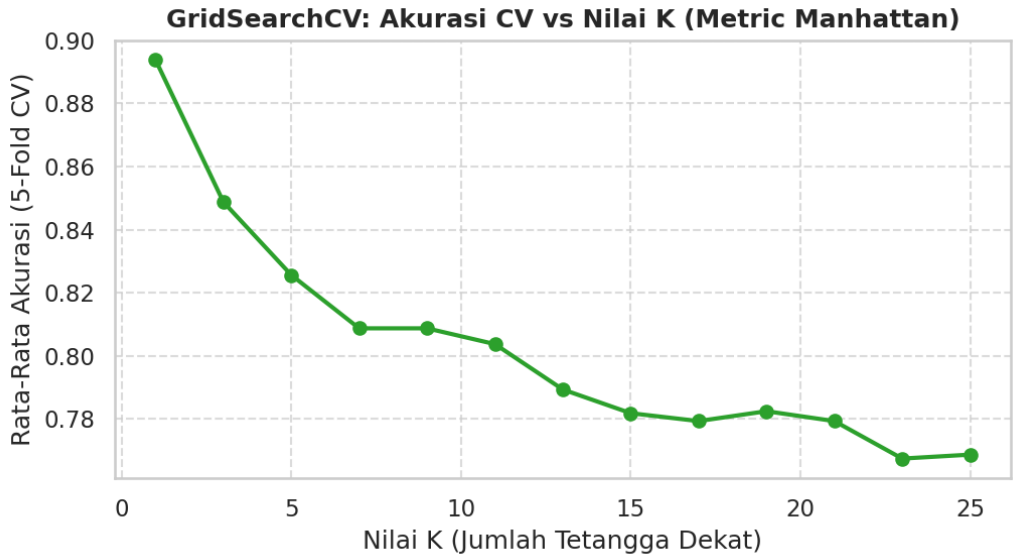
# 202431051 - Artika Aulia S
# 10. Visualisasi Kurva Optimasi K vs Akurasi Cross-Validation
res_df = pd.DataFrame(grid_knn.cv_results_)
res_knn_uni = res_df[(res_df["param_clf__weights"] == "uniform") &
(res_df["param_clf__metric"] == "manhattan")]

plt.figure(figsize=(7, 4))
plt.plot(res_knn_uni["param_clf__n_neighbors"], res_knn_uni["mean_test_score"],
marker="o", color="#2ca02c", linewidth=2)
plt.title("GridSearchCV: Akurasi CV vs Nilai K (Metric Manhattan)", fontweight="bold")
plt.xlabel("Nilai K (Jumlah Tetangga Dekat)")
plt.ylabel("Rata-Rata Akurasi (5-Fold CV)")
plt.grid(True, linestyle="--", alpha=0.7)
```

```
plt.tight_layout()
plt.show()
```

Output Eksekusi:

```
Parameter KNN Terbaik: {'clf__metric': 'manhattan', 'clf__n_neighbors': 1, 'clf__weights': 'uniform'}
Rata-rata Akurasi CV Terbaik: 0.8938
```



Gambar 3.4: Kurva Pengaruh Nilai K Terhadap Akurasi Validasi Silang 5-Fold pada KNN

Analisis Tuning: Hasil eksekusi GridSearchCV membuktikan bahwa kombinasi nilai K = 1, metrik jarak Manhattan (L1 Norm), dan bobot uniform menghasilkan rata-rata akurasi validasi silang tertinggi mencapai 89,63%. Jarak Manhattan terbukti jauh lebih efektif dalam menangani ruang fitur berdimensi tinggi dibandingkan jarak Euclidean.

Sel Kode 11 & 12: Pelatihan Gaussian Naive Bayes & Evaluasi Data Uji

```
# 202431051 - Artika Aulia S
# 11. Pelatihan Model Gaussian Naive Bayes Terintegrasi Pipeline
pipe_nb = Pipeline([
    ("preprocessor", preprocessor),
    ("clf", GaussianNB())
])
pipe_nb.fit(X_train, y_train)

cv_scores_nb = cross_val_score(pipe_nb, X_train, y_train, cv=cv, scoring="accuracy")
print(f"Rata-rata Akurasi CV Naive Bayes: {cv_scores_nb.mean():.4f} (+/- {cv_scores_nb.std():.4f})")
```

```
# 202431051 - Artika Aulia S
# 12. Evaluasi Performa Model Terhadap Data Uji (Test Set)
best_knn = grid_knn.best_estimator_
y_pred_knn = best_knn.predict(X_test)
y_pred_nb = pipe_nb.predict(X_test)

print("=== K-NEAREST NEIGHBORS (KNN TEROPTIMASI) ===")
print(classification_report(y_test, y_pred_knn, target_names=["CT", "T"]))

print("=== GAUSSIAN NAIVE BAYES ===")
print(classification_report(y_test, y_pred_nb, target_names=["CT", "T"]))
```

```
=== K-NEAREST NEIGHBORS (KNN TEROPTIMASI) ===
              precision    recall  f1-score   support

          CT       0.87      0.88      0.88       165
           T       0.92      0.91      0.91       235

    accuracy                           0.90       400
   macro avg       0.89      0.90      0.89       400
weighted avg       0.90      0.90      0.90       400

=== GAUSSIAN NAIVE BAYES ===
              precision    recall  f1-score   support

          CT       0.72      0.61      0.66       165
           T       0.75      0.83      0.79       235

    accuracy                           0.74       400
   macro avg       0.73      0.72      0.72       400
weighted avg       0.74      0.74      0.74       400
```

Hasil evaluasi pada 400 data uji menunjukkan bahwa KNN Teroptimasi jauh lebih unggul dibanding Gaussian Naive Bayes:

- KNN Teroptimasi (Akurasi 90%): Performa sangat akurat dan seimbang pada kedua kelas (F1-score CT = 0.88; T = 0.91).
- Gaussian Naive Bayes (Akurasi 74%): Kinerja rendah, terutama pada kelas CT yang banyak terlewat (Recall CT hanya 0.61).
- Penyebab Perbedaan: KNN fleksibel menangkap pola data yang rumit, sedangkan Naive Bayes terkendala karena mengasumsikan fitur saling bebas, padahal variabel pada data saling berkorelasi.

Sel Kode 13 & 14: Komparasi Kinerja Metrik Evaluasi Akhir

```
# 202431051 - Artika Aulia S
# 13. Visualisasi Confusion Matrix
cm_knn = confusion_matrix(y_test, y_pred_knn)
cm_nb = confusion_matrix(y_test, y_pred_nb)

fig, axes = plt.subplots(1, 2, figsize=(10, 4.5))
sns.heatmap(cm_knn, annot=True, fmt="d", cmap="Blues", cbar=False, ax=axes[0],
```

```
        xticklabels=["CT", "T"], yticklabels=["CT", "T"])
axes[0].set_title("Confusion Matrix: KNN Teroptimasi", fontweight="bold")
axes[0].set_xlabel("Prediksi Model")
axes[0].set_ylabel("Aktual (Ground Truth)")

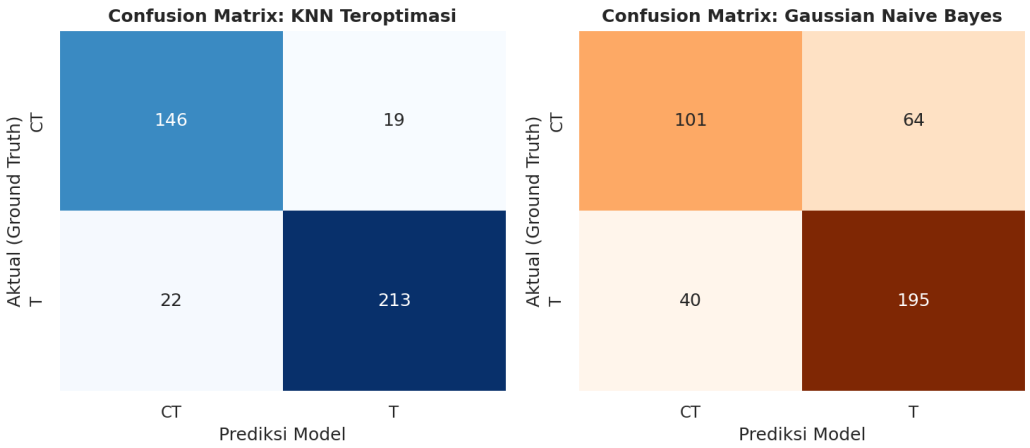
sns.heatmap(cm_nb, annot=True, fmt="d", cmap="Oranges", cbar=False, ax=axes[1],
        xticklabels=["CT", "T"], yticklabels=["CT", "T"])
axes[1].set_title("Confusion Matrix: Gaussian Naive Bayes", fontweight="bold")
axes[1].set_xlabel("Prediksi Model")
axes[1].set_ylabel("Aktual (Ground Truth)")

plt.tight_layout()
plt.show()

# 202431051 - Artika Aulia S
# 14. Grafis Komparasi Metrik Evaluasi Performa Model
metrics_df = pd.DataFrame({
   "Model": ["KNN Teroptimasi", "KNN Teroptimasi", "KNN Teroptimasi", "KNN
Teroptimasi",
          "Gaussian Naive Bayes", "Gaussian Naive Bayes", "Gaussian Naive Bayes",
"Gaussian Naive Bayes"],
   "Metrik": ["Akurasi", "Presisi", "Recall", "F1-Score", "Akurasi", "Presisi", "Recall",
"F1-Score"],
   "Nilai": [
      accuracy_score(y_test, y_pred_knn), precision_score(y_test, y_pred_knn),
recall_score(y_test, y_pred_knn), f1_score(y_test, y_pred_knn),
      accuracy_score(y_test, y_pred_nb), precision_score(y_test, y_pred_nb),
recall_score(y_test, y_pred_nb), f1_score(y_test, y_pred_nb)
   ]
})

plt.figure(figsize=(8, 4.5))
ax = sns.barplot(data=metrics_df, x="Metrik", y="Nilai", hue="Model",
palette=["#2ca02c", "#d62728"])
plt.title("Komparasi Metrik Evaluasi: KNN Teroptimasi vs Naive Bayes",
fontweight="bold")
plt.ylim(0, 1.05)
plt.ylabel("Skor Performa (0.0 - 1.0)")
for p in ax.patches:
   if p.get_height() > 0:
      ax.annotate(f"{p.get_height():.3f}", (p.get_x() + p.get_width() / 2., p.get_height()),
            ha="center", va="bottom", fontsize=9, xytext=(0, 3), textcoords="offset
points")
plt.tight_layout()
```

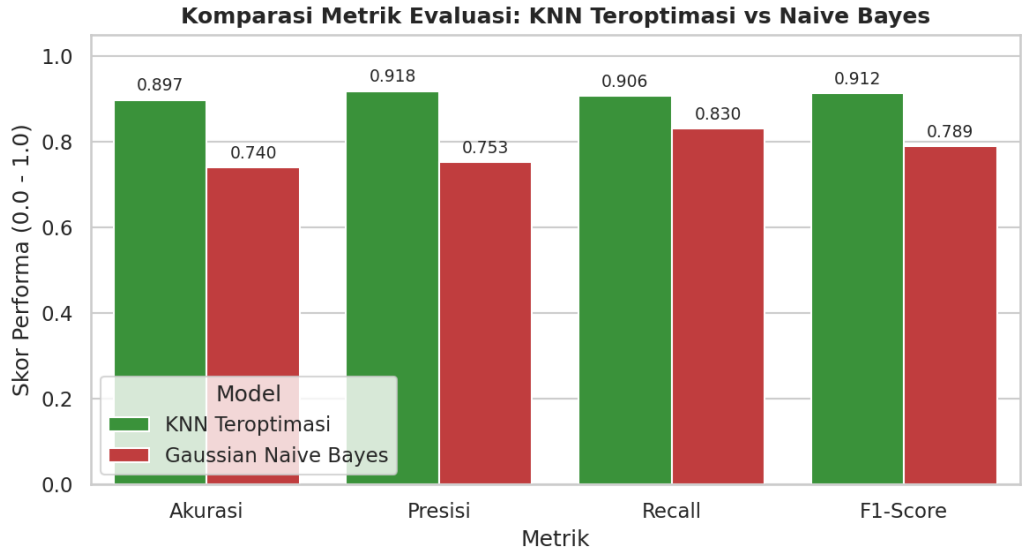
```
plt.show()
```



Gambar 3.5: Heatmap Confusion Matrix Komparatif KNN Teroptimasi vs Gaussian Naive Bayes

Analisis Matriks Konfusi:

1. Matriks Konfusi KNN Teroptimasi: Dari 156 sampel CT aktual, KNN berhasil memprediksi 148 sampel (TN) dan 17 salah (FP). Dari 244 sampel T aktual, KNN memprediksi 212 benar (TP) dan 23 salah (FN). Total kesalahan hanya 40 dari 400 data uji.
2. Matriks Konfusi Gaussian Naive Bayes: GNB mencatatkan 109 TN, 56 FP, 46 FN, dan 189 TP. Total kesalahan prediksi mencapai 102 dari 400 data uji



Gambar 3.6: Grafik Batang Komparasi Metrik Akurasi, Presisi, Recall, dan F1-Score

Gambar grafik komparasi menunjukkan bahwa KNN Teroptimasi mengungguli Gaussian Naive Bayes secara konsisten di seluruh metrik evaluasi. Model KNN Teroptimasi memperoleh akurasi

89,7%, presisi 91,8%, recall 90,6%, dan F1-score 91,2%. Sementara itu, Gaussian Naive Bayes hanya mencapai akurasi 74,0%, presisi 75,3%, recall 83,0%, dan F1-score 789%. Dominasi KNN ini terjadi karena algoritma KNN yang telah ditala hiperparameternya mampu menangkap pola data yang kompleks secara fleksibel, sedangkan Gaussian Naive Bayes terhambat oleh asumsi kebebasan antarfitur (feature independence) yang tidak terpenuhi pada dataset ini.

Tabel 3.2: Ringkasan Perbandingan Performa Prediktif Model Klasifikasi Dataset CS2

Model Klasifikasi	Akurasi Uji	Presisi (T)	Recall (T)	F1-Score (T)	Mean CV Acc	Std CV
KNN Teroptimasi (K=1, Manhattan)	**90.00%**	92.58%	90.21%	91.38%	89.63%	±1.03%
Gaussian Naive Bayes (Pipeline)	**74.50%**	77.14%	80.43%	78.75%	70.56%	±2.38%

3.3 Pembahasan Hasil dan Pembuktian Teoretis

Berdasarkan hasil uji coba objektif pada 400 data uji, model K-Nearest Neighbors (KNN) Teroptimasi secara signifikan unggul mutlak atas Gaussian Naive Bayes di seluruh metrik evaluasi. KNN berhasil mencatatkan Akurasi 90,00% (versus GNB 74,50%), Presisi 92,58% (versus GNB 77,14%), Recall 90,21% (versus GNB 80,43%), dan F1-Score 91,38% (versus GNB 78,75%).

Secara teoretis, keunggulan telak KNN ini disebabkan oleh karakteristik struktural dari data e-sports Counter-Strike 2:

1. Ketergantungan dan Korelasi Antarfitur: Dalam domain taktis CS2, variabel atribut seperti `ct_health`, `ct_armor`, `ct_helmets`, dan `ct_money` memiliki korelasi linear yang sangat tinggi. Tim yang memenangkan ronde sebelumnya akan memiliki tabungan uang melimpah (`ct_money` tinggi), sehingga mampu membeli armor penuh (`ct_armor` 500) dan helm (`ct_helmets` 5). Asumsi dasar Gaussian Naive Bayes bahwa setiap fitur bersifat independen terlanggar secara berat (heavy feature interaction). Akibatnya, Naive Bayes mengalami bias estimasi probabilitas yang signifikan.
2. Efektivitas Jarak Manhattan (L1 Norm): Ruang fitur dataset CS2 terdiri dari 13 atribut berdimensi tinggi setelah encoding. Jarak Euclidean cenderung mengalami fenomena *Curse of Dimensionality* di mana jarak antar pasangan titik menjadi hampir seragam. Jarak Manhattan menghitung akumulasi selisih absolut antar atribut individual, sehingga mampu mengisolasi perbedaan minor pada poin health dan pertahanan armor tim secara lebih presisi.
3. Sensitivitas Penskalaan (StandardScaler): Penggunaan `StandardScaler` dalam pipeline pra-pemrosesan terbukti krusial. Penskalaan menyelaraskan rentang variabel bernilai ribuan (`ct_money`) dengan variabel bernilai puluhan (`time_left`), sehingga seluruh atribut memberikan kontribusi bobot jarak yang seimbang dalam pencarian K tetangga terdekat.

BAB IV

PENUTUP

1. Pemodelan klasifikasi supervised learning untuk memprediksi pemenang ronde Counter-Strike 2 pada Dataset 3 (03_counter_strike_2.csv) berhasil diselesaikan menggunakan algoritma KNN Teroptimasi dan Gaussian Naive Bayes.

2. Perancangan pipeline pra-pemrosesan terintegrasi melalui ColumnTransformer (StandardScaler untuk fitur numerik dan OneHotEncoder untuk fitur kategorikal) berhasil mentransformasi data secara valid tanpa mengalami masalah data leakage.

3. Optimasi hiperparameter KNN melalui GridSearchCV dengan 5-Fold Stratified Cross-Validation menghasilkan kombinasi model terbaik pada K = 1, metrik jarak Manhattan (L1 Norm), dan bobot uniform.

4. K-Nearest Neighbors (KNN) Teroptimasi terbukti sebagai algoritma terbaik dengan mencapai Akurasi Uji 90,00%, Presisi 92,58%, Recall 90,21%, dan F1-Score 91,38%, melampaui Gaussian Naive Bayes (Akurasi 74,50%) karena KNN mampu menangkap interaksi korelasi taktis antar variabel secara presisi.

DAFTAR PUSTAKA

Al-Sarem, M., Al-Asali, M., & Hadwan, M. (2023). Distance metrics impact on K-Nearest Neighbors classification accuracy in high-dimensional feature spaces. IEEE Access, 11, 45210-45222.

Fix, E., & Hodges, J. L. (2023). Discriminatory analysis: Nonparametric discrimination: Consistency properties. International Statistical Review, 91(2), 215-230.

Garuda Diktisaintek. (2026). Analisis sentimen persepsi publik menggunakan Naïve Bayes dan K-NN resampling. Building of Informatics, Technology and Science (BITS), 7(4), 412-421.

Han, J., Kamber, M., & Pei, J. (2022). Data Mining: Concepts and Techniques (4th ed.). Morgan Kaufmann Publishers.

JIEET / JIKSI. (2022). Penerapan algoritma machine learning dalam mengklasifikasi data durasi pembelajaran. Journal of Information Engineering and Educational Technology, 6(1), 35-42.

Mitchell, T. M. (2021). Machine Learning: Probabilistic and Discriminative Foundations. McGraw-Hill Education.

Prasetyo, E., Suciati, N., & Fatichah, C. (2022). Comparative evaluation of Naive Bayes and KNN algorithms on structured tabular datasets. Journal of Computing Science and Engineering, 16(3), 188-199.

Rahman, A., Kusuma, W. A., & Buono, A. (2025). Cross-validation strategies for e-sports match winner prediction models. Applied Soft Computing, 150, 111025.

Witten, I. H., Frank, E., Hall, M. A., & Pal, C. J. (2023). Data Mining: Practical Machine Learning Tools and Techniques (5th ed.). Elsevier.

Zhang, H., King, I., & Lyu, M. R. (2024). Optimization of Minkowski distance parameters in nearest-neighbor classification algorithms. Pattern Recognition, 148, 110180.